Appendix

A.1. Required Software

The numerical workflow developed in this thesis combines several open-source tools for geometry generation, finite element simulation, and post-processing. The software required to reproduce the results is listed below:

- **Gmsh**: generation of the computational geometry and structured quadrilateral mesh.
- **MeshIO**: conversion of the mesh from Gmsh (.msh) format to XDMF format compatible with FEniCSx.
- **Python 3**: programming environment used for the implementation of the numerical model.
- **FEniCSx**: finite element framework used to solve the governing equations.
- **MPI4Py and PETSc**: libraries employed for parallel computation and linear system solution.
- **Visual Studio Code**: development environment used to edit and execute the simulation scripts.
- **ParaView**: post-processing software used to visualize the temperature, liquid fraction, pressure, and velocity fields generated during the simulations.

A.2. Study Case

The study case considered in this thesis consists of the thermal analysis of a battery cell surrounded by a phase change material (PCM). The objective is to investigate the heat transfer between the battery and the PCM, as well as the evolution of the melting process induced by the heat generated during battery operation.

The computational domain is composed of a square enclosure filled with a water–ice PCM and a rectangular battery located at its center. A constant volumetric heat generation term is applied within the battery, representing the thermal energy produced during operation. Initially, the entire domain is at a uniform temperature below the melting temperature of water, meaning that the PCM is completely solid.

The phase transition is modeled using the enthalpy method, which incorporates latent heat effects into the energy equation through an apparent heat-capacity formulation. As the battery releases heat, the surrounding PCM absorbs thermal energy and progressively melts, acting as a passive thermal storage system capable of delaying temperature increases within the battery.

A.3. Geometry and Mesh Generation in Gmsh

The script is uploaded in the Github repository

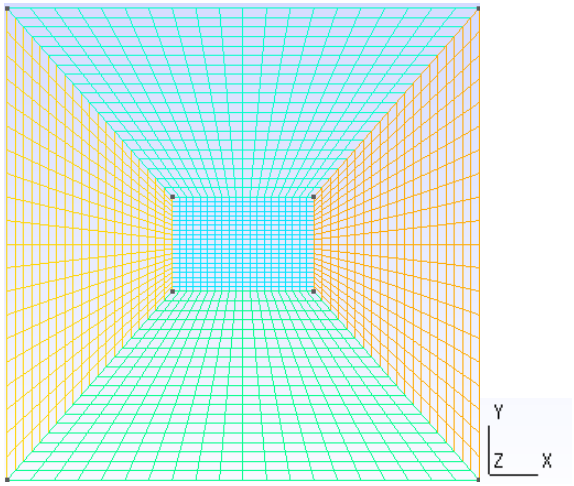

Figure 1: Gmsh mesh of the battery geometry for the simulation

The computational geometry was generated in Gmsh using a .geo script. The domain consists of a square enclosure representing the PCM region and a centered rectangular battery. The geometry is divided into four PCM subdomains surrounding the battery, allowing the use of a structured meshing strategy.

A transfinite discretization is applied to all surfaces and subsequently recombined into quadrilateral elements. This approach provides a regular mesh distribution and improves numerical accuracy and stability compared with triangular elements. Finally, physical groups are assigned to the PCM, battery, and boundary regions for their later identification in FEniCSx.

A.4. Mesh Conversion to XDMF

The python code is uploaded in the GitHub repository.

After generating the mesh in Gmsh, the .msh file is converted into XDMF format using the meshio library. This step is necessary to make the mesh compatible with the FEniCSx workflow.

The script first reads the original Gmsh mesh and separates the quadrilateral elements, which define the computational domain, from the line elements, which represent the boundary entities. The corresponding physical tags are preserved during the conversion, allowing FEniCSx to identify the different regions and boundaries of the problem.

Two output files are generated: mesh_quad.xdmf, containing the quadrilateral domain mesh, and mf_quad.xdmf, containing the boundary markers. These files are then used as input for the numerical simulation.

A.5. Python Implementation

Importing libraries

```
1   import numpy as np
2   from mpi4py import MPI
3   from petsc4py import PETSc
4   import ufl
5
6   from dolfinx import fem
7   from dolfinx.io import gmsh, VTXWriter
8   from dolfinx.fem.petsc import LinearProblem
9   from basix.ufl import element
10
11  comm = MPI.COMM_WORLD
12  rank = comm.rank
13  TAG = (lambda s: print(s, flush=True)) if rank == 0 else (lambda s: None)
```

The first section of the code imports the libraries required for the simulation. NumPy is used for numerical operations, while MPI4Py and PETSc provide parallel computing and linear solver capabilities. The UFL library is employed to define the variational formulation of the governing equations.

The finite element framework is provided by FEniCSx, which handles mesh operations, function spaces, input/output management, and the solution of the problem. Finally, the MPI communicator and processor rank are initialized, enabling the code to run in both serial and parallel environments.

Mesh import

```
ret = gmsh.read_from_msh("battery_pcm_quad.msh", comm, rank=0, gdim=2)
domain = ret[0]

x = domain.geometry.x
xmin, ymin = float(np.min(x[:, 0])), float(np.min(x[:, 1]))
xmax, ymax = float(np.max(x[:, 0])), float(np.max(x[:, 1]))

TAG("Quadrilateral Gmsh mesh imported")
TAG(f"Domain size = {xmax-xmin:.4f} x {ymax-ymin:.4f} m")

dx = ufl.Measure("dx", domain=domain)
tdim = domain.topology.dim
```

In this section, the quadrilateral mesh generated in Gmsh is imported into FEniCSx using the read_from_msh() function. Once the mesh is loaded, the computational domain is created and its geometric coordinates are extracted. The minimum and maximum coordinates are then used to determine the dimensions of the domain and verify that the mesh has been imported correctly.

Finally, the integration measure dx is defined, allowing the finite element integrals to be evaluated over the entire domain in the variational formulation of the problem.

Geometry definition

```
L = 0.10
bx = 0.03
by = 0.02
cx = L / 2
cy = L / 2

batt_xmin = cx - bx / 2
batt_xmax = cx + bx / 2
batt_ymin = cy - by / 2
batt_ymax = cy + by / 2

tol = 1e-10

def inside_battery(x):
    return ( (x[0] >= batt_xmin - tol)
           & (x[0] <= batt_xmax + tol)
           & (x[1] >= batt_ymin - tol)
           & (x[1] <= batt_ymax + tol) )
```

This section defines the geometry of the problem. The computational domain consists of a square enclosure with side length L= 0.10 m, while the battery is represented by a centered rectangular region with dimensions 0.03 x 0.02 m. The battery limits are computed from the coordinates of its center, allowing its position to be defined independently of the mesh.

The function inside_battery() is then introduced to identify whether a point belongs to the battery region. This function is later used to assign different material properties and the internal heat generation term to the battery domain.

Material properties

```
56    rho_pcm = 997.0
57    cp_pcm = 4182.0
58    k_pcm = 0.6
59
60    rho_batt = 2500.0
61    cp_batt = 900.0
62    k_batt = 5.0
63
64    Q_batt = 5.0e6
65
66    Tm = 273.15
67    Lh = 150000.0
68    delta_T = 0.5
```

This section defines the thermophysical properties of the two materials present in the simulation: the water–ice PCM and the battery. For each material, the density, specific heat capacity, and thermal conductivity are specified. In addition, a uniform volumetric heat generation term is assigned to the battery to represent the heat produced during operation.

The parameters governing the phase-change process are also defined, including the melting temperature, the latent heat of fusion, and the temperature interval used to smooth the solid–liquid transition. These properties are subsequently employed in the enthalpy formulation to model the melting of the PCM.

Time discretization

```
73    dt = 0.05
74    t_end = 300.0
75    nsteps = int(round(t_end / dt))
76    save_every = 20     # saves every 1 s
```

This section defines the temporal parameters of the simulation. The time step is set to $\Delta t = 0.05$ s, while the total simulation time is fixed at 300 s. From these values, the total number of time steps is automatically calculated. In addition, the results are configured to be saved every 20 iterations, corresponding to an output frequency of 1 s.

Function spaces and initial conditions

```
81    cell = domain.ufl_cell().cellname()
82
83    Te = element("Lagrange", cell, 1)
84    Tspace = fem.functionspace(domain, Te)
85
86    Vvec = fem.functionspace(domain, element("Lagrange", cell, 1, shape=(tdim,)))
87
88    T = ufl.TrialFunction(Tspace)
89    s = ufl.TestFunction(Tspace)
90
91    Tn = fem.Function(Tspace, name="Temperature")
92    Tn.interpolate(lambda x: np.full(x.shape[1], 268.15))
93    Tn.x.scatter_forward()
94
95    T_viz = fem.Function(Tspace, name="Temperature")
96    fl_viz = fem.Function(Tspace, name="LiquidFraction")
97    battery_viz = fem.Function(Tspace, name="BatteryRegion")
98    u_zero_viz = fem.Function(Vvec, name="Velocity")
99
100   u_zero_viz.x.array[:] = 0.0
101   u_zero_viz.x.scatter_forward()
```

This section defines the finite element spaces used in the simulation. A first-order Lagrange element is adopted for the temperature field, and the corresponding function space is created over the computational domain. The trial and test functions required for the variational formulation are also defined.

The initial temperature field is then assigned a uniform value of 268.15 K throughout the domain, which is below the melting temperature of water. Consequently, the PCM is initially in a fully solid state. Additional functions are created for post-processing purposes, allowing the visualization of temperature, liquid fraction, battery region, and velocity fields in ParaView.

Region markers

```
battery_marker = fem.Function(Tspace, name="BatteryMarker")
battery_marker.interpolate(lambda x: np.where(inside_battery(x), 1.0, 0.0))
battery_marker.x.scatter_forward()

pcm_marker = fem.Function(Tspace, name="PCMMarker")
pcm_marker.x.array[:] = 1.0 - battery_marker.x.array
pcm_marker.x.scatter_forward()

TAG(f"Initial Tmin = {np.min(Tn.x.array):.2f} K")
TAG(f"Initial Tmax = {np.max(Tn.x.array):.2f} K")
TAG(f"Battery marker min/max = {np.min(battery_marker.x.array):.1f}/{np.max(battery_marker.x.array):.1f}")
```

This section defines the regions occupied by the battery and the PCM within the computational domain. A marker function is created to identify the battery cells, assigning a value of 1 inside the battery and 0 elsewhere. The PCM marker is then obtained as the complementary region of the battery marker.

These marker functions are subsequently used to assign different material properties and apply the volumetric heat generation term exclusively within the battery region. Finally, diagnostic messages are printed to verify the correct initialization of the temperature field and the region markers.

Output update

```
def update_output():
    T_viz.x.array[:] = Tn.x.array
    T_viz.x.scatter_forward()

    fl_viz.x.array[:] = 0.5 * (
        1.0 + np.tanh((Tn.x.array - Tm) / delta_T)
    )
    fl_viz.x.scatter_forward()

    battery_viz.x.array[:] = battery_marker.x.array
    battery_viz.x.scatter_forward()
```

This section defines the function responsible for updating the variables exported to ParaView during the simulation. The current temperature field is copied to the visualization variable, while the liquid fraction is computed using the smoothed hyperbolic tangent formulation adopted in the enthalpy method.

The battery marker is also updated, allowing the battery region to be visualized together with the temperature and liquid fraction fields. These quantities are subsequently written to the output file at the specified time intervals.

Time loop and enthalpy method

```
141  TAG("Starting water/ice PCM simulation on quadrilateral battery mesh")
142
143  with VTXWriter( domain.comm, "results_water_ice_quad_pcm.bp",
144      [T_viz, fl_viz, battery_viz, u_zero_viz],)
145      as vtx:
146
147      update_output()
148      vtx.write(0.0)
149
150      for step in range(1, nsteps + 1):
151
152          t = step * dt
153
154          # Liquid fraction
155          fl_n = 0.5 * (1.0 + ufl.tanh((Tn - Tm) / delta_T))
156
157          # Apparent heat capacity
158          dfl_dT = 0.5 / delta_T * (  1.0 - ufl.tanh((Tn - Tm) / delta_T) ** 2 )
159          cp_eff_pcm = cp_pcm + Lh * dfl_dT
160
161          rho_cp_eff = (
162              pcm_marker * rho_pcm * cp_eff_pcm  + battery_marker * rho_batt * cp_batt )
163
164          k_eff = ( pcm_marker * k_pcm + battery_marker * k_batt )
```

This section contains the main simulation loop. At each time step, the liquid fraction is updated using the smoothed enthalpy formulation, allowing the phase state of the PCM to evolve continuously between solid and liquid conditions. The derivative of the liquid fraction with respect to temperature is then used to compute the apparent heat capacity, which incorporates the latent heat effect into the energy equation.

Using the material markers defined previously, the effective thermophysical properties are assigned separately to the PCM and battery regions. This approach enables the simulation of heat generation within the battery while simultaneously accounting for the melting process occurring in the surrounding PCM.

Energy equation and solver

```
        a_T = ( (rho_cp_eff / dt) * T * s * dx + k_eff * ufl.inner(ufl.grad(T), ufl.grad(s)) * dx)

        L_T = ( (rho_cp_eff / dt) * Tn * s * dx + battery_marker * Q_batt * s * dx)

        problem_T = LinearProblem( a_T, L_T, bcs=[],
            petsc_options_prefix=f"water_ice_quad_{step}_",
            petsc_options=opts_temp,  )

        T_out = problem_T.solve()
        T_out.x.scatter_forward()

        if np.any(np.isnan(T_out.x.array)):
            raise RuntimeError(f"NaN detected at t={t:.2f}s")

        Tn.x.array[:] = T_out.x.array
        Tn.x.scatter forward()
```

This section defines and solves the transient energy equation using the finite element method. The variational formulation incorporates the effective heat capacity previously computed through the enthalpy method, allowing the latent heat effects to be included in the simulation. The heat generation term is applied only within the battery region through the battery marker function.

After assembling the system, the linear problem is solved using PETSc solvers. The resulting temperature field is then updated and checked to ensure numerical stability throughout the simulation.

Saving results

```
        if step % save_every == 0 or step == nsteps:

            update_output()
            vtx.write(t)

            Tmin = float(np.min(Tn.x.array))
            Tmax = float(np.max(Tn.x.array))
            dT = Tmax - Tmin

            fl_mean = float(np.mean(fl_viz.x.array))
            fl_max = float(np.max(fl_viz.x.array))

            TAG(
                f"t={t:7.2f}s | "
                f"Tmin={Tmin:7.2f} K | "
                f"Tmax={Tmax:7.2f} K | "
                f"ΔT={dT:7.2f} K | "
                f"fl_mean={fl_mean:5.3f} | "
                f"fl_max={fl_max:5.3f}"
            )

    TAG("Simulation completed")
    TAG("Saved as results_water_ice_quad_pcm.bp")
```

This final section manages the storage of simulation results and the monitoring of key variables during the computation. At the specified output intervals, the temperature field, liquid fraction, and battery region are updated and written to the output file for subsequent visualization in ParaView.

Additionally, diagnostic quantities such as the minimum and maximum temperature, temperature range, and liquid fraction statistics are computed and displayed in the terminal. These values provide a simple way to monitor the progression of the melting process and verify the stability of the simulation. Once the time loop is completed, a final message confirms the successful execution of the simulation and the creation of the output file.

Visualization and Post-Processing in ParaView

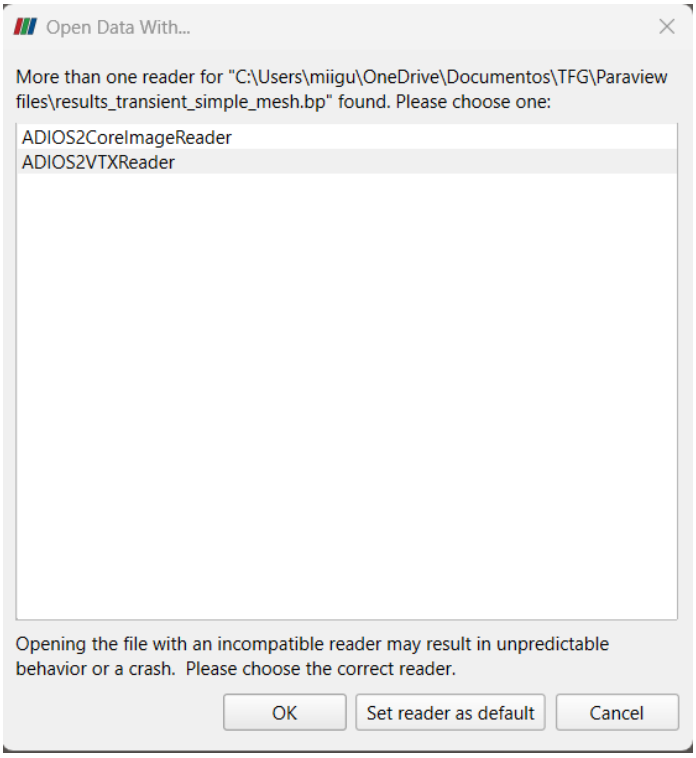

Figure 2: Selection of the ADIOS2VTXReader in ParaView for the visualization of transient results exported from FEniCSx in .bp format.

The results obtained from the FEniCSx simulations were visualized using ParaView. During the development of the project, it was observed that exporting the results in the traditional XDMF format generated compatibility issues when handling multiple transient fields and large datasets.

For this reason, the output was written using the ADIOS2-based .bp format through the VTXWriter functionality available in FEniCSx.

Once the simulation is completed, the generated .bp file can be opened directly in ParaView. When loading the file, ParaView may detect multiple compatible readers and prompt the user to select one. In this work, the ADIOS2VTXReader was used, as it provides full compatibility with the data structure generated by FEniCSx.

The .bp format allows efficient storage of transient data and supports the visualization of multiple fields throughout the simulation. Using ParaView, it is possible to inspect the evolution of temperature, liquid fraction, battery region markers, and velocity fields, as well as generate animations, contour plots, streamlines, and quantitative analyses of the simulation results.

The general post-processing procedure is:

1. Open the generated .bp file in ParaView.
2. Select ADIOS2VTXReader as the file reader.
3. Apply the reader and load the available fields.
4. Choose the desired variable (Temperature, Liquid Fraction, Velocity, etc.).
5. Use the time controls to visualize the transient evolution of the solution.
6. Generate screenshots, animations, or additional derived quantities for analysis.

A.13 Appendix Conclusions

This appendix has presented the complete workflow required to reproduce the battery–PCM simulation developed in this thesis. Starting from the generation of the computational geometry and mesh in Gmsh, the procedure includes the conversion of the mesh to a format compatible with FEniCSx, the implementation of the enthalpy method for phase-change modeling, the execution of the numerical simulation, and the post-processing of the results in ParaView.

The step-by-step description of the code and numerical setup provides a practical guide for future users interested in reproducing, validating, or extending the present work. Together with the corresponding project files and GitHub repository, this appendix aims to facilitate the development of more advanced simulations involving phase-change materials, natural convection, and battery thermal management applications.